

AI Wardrobe (AI Outfitter)

Backend API Audit Report

App: AI Wardrobe / AI Outfitter

Audit date: Sep 17, 2026

Purpose: Hand this document to a backend/fullstack developer to build required APIs.

Executive Summary

Integration	Status
Custom REST API (ApiClient)	Scaffolded only — API_BASE_URL defaults to https://api.example.com; all remote datasources return empty stubs
BFL (FLUX) API	Live — POST https://api.bfl.ai/v1/flux-2-pro + polling URL
Firebase Auth	Live — email, Google, guest
Firebase Storage	Live — profile avatars users/{uid}/avatar.jpg
Cloud Firestore	Live (optional) — FCM token on users/{uid}
Catalogs / Home / Wardrobe	Hardcoded — StyleCatalogStore, WardrobeCatalog, AppAssets
History / My Wardrobe	Local only — SharedPreferences + app file storage
Settings / Language / Onboarding	Local only — SharedPreferences

Auth for future REST: ApiClient sends `Authorization: Bearer {API_KEY}` from `.env`. User-scoped endpoints should also verify Firebase ID token (`Authorization: Bearer {firebaseIdToken}`).

Screen-by-Screen API Specification

#	Screen	Purpose	Current Source	Data Fields	API Needed?	Type	Suggested Endpoint	Assets	Auth?	Notes / Edge Cases
1	Splash	Boot, precache hero, route to onboarding/language/login/home	Local prefs + Firebase (non-blocking)	onboardingCompleted, hasSelectedLanguage, isSignedIn, boot readiness	No	—	—	assets/images/splash/splash_hero.png, Lottie assets/animation/loading.json (local)	No	12s boot timeout; auth read has 6s timeout. No backend call today.
2	Onboarding	3-page intro; mark complete	Hardcoded pages + SharedPreferences	onboarding_completed, page title, body, imageUrl	Optional	GET	/api/v1/onboarding/pages	assets/images/onboarding/* (3 PNGs, local)	No	CMS-driven pages optional. Today fully bundled.
3	Login	Email/password, Google, guest sign-in	Firebase Auth	email, password, rememberMe → id, email, displayName, photoUrl, isGuest	No*	—	Firebase Auth SDK	assets/icons/ic_email.png, ic_lock.png, ic_google.png, ic_guest.png (local)	Produces session	*No custom REST; Firebase handles auth. Optional: POST /api/v1/auth/sync-session after login.
4	Sign Up	Register + optional profile photo	Firebase Auth + Firebase Storage + local file	name, email, password, photoPath → AuthUser fields	No*	—	Firebase Auth + Storage	ic_user, ic_add_profile, auth icons (local)	Produces session	Photo compressed client-side → users/{uid}/avatar.jpg in Firebase Storage.
5	Forgot Password	Send reset email	Firebase Auth	email	No	—	Firebase sendPasswordResetEmail	ic_email.png (local)	No	Resend supported.
6	New Password	Update password (signed-in)	Firebase Auth	email, password, confirmPassword	No	—	Firebase updatePassword	ic_lock.png (local)	Yes	requires-recent-login if session stale.
7	Home	AI Studio banner, Beauty Lab, Occasions, Couple/Duo rails	Hardcoded widgets; homeProvider → empty HomeFeed stub	sections[]: id, titleKey, categoryId, sortOrder, items[]: styleId, thumbnailUrl, label, genderScope	Yes	GET	/api/v1/home/feed?gender=men women	Studio/banner PNGs in assets/images/home/ui/; rail previews from bundled catalog PNGs (local)	Optional	Provider shows shimmer then renders hardcoded UI regardless of empty feed. Pagination per rail recommended. Gender from settings_style_gender pref.
8	Wardrobe	Regional traditional categories (Chinese, Indian, Korean, Pakistani, Arabian)	Hardcoded WardrobeCatalog + StyleCatalogStore previews	categories[]: id, titleKey, browseTabId, backgroundToken, previewItems[]: styleId, thumbnailUrl, gender	Yes	GET	/api/v1/wardrobe/categories?gender=	assets/images/wardrobe/*, regional catalog PNGs under assets/images/home/(men,women)/wardrobe/regional/ (local, ~500+ images)	No	Gender filter from SharedPreferences. Remote stub exists but unused by UI.
9	Style Catalog	Full style grid per category (try-on, hair, beard, hijab, occasions, couple, presets, wardrobe browse)	Hardcoded StyleCatalogStore.byId(categoryId)	StyleCatalog: categoryId, titleKey, genderScope, genderTabs[], tabs[], items[]: id, tabId, genderTabId, imageUrl, promptCommand, sortOrder	Yes	GET	/api/v1/catalog/{categoryId}?tab=&gender=	Hundreds of bundled PNGs in assets/images/home/** (local today; should become CDN URLs)	No	Tabs: garment types, regions, occasions. No pagination — all items loaded at once today.
10	Try-On	Photo upload → edit/mask → AI generate → result → save/share/favorite	BFL API (live) + local ML Kit + local history save	Input: sourceImage, styleId, categoryId, mode, prompt, styleReferenceImage. Output: pollingUrl, status, resultImageUrl, error	Yes (proxy)	POST + GET	POST /api/v1/try-on/generate, GET /api/v1/try-on/jobs/{jobId}	Style refs from catalog assets (local) copied to temp; result from BFL URL (network)	API key / user token	Today calls BFL directly with .env API_KEY. Backend should proxy BFL, store jobs, return signed result URLs. Poll every 2s. Cancel supported. Auto-saves to local history on success.
11	History	All generated looks; multi-select delete	Local SharedPreferences (history_items); remote stub []	id, createdAt, imageUrl, sourceImageUrl, isFavorite	Yes	GET, DELETE	GET /api/v1/history?page=&limit=, DELETE /api/v1/history (bulk)	Result images: local file paths or remote URLs (network after gen)	Yes (for sync)	Merge: remote wins on ID collision. Deletes local image files. Offline cache recommended.
12	Favorites	Filtered history (isFavorite == true)	Same as History (local filter)	Same as History	Yes	GET, PATCH	GET /api/v1/history?favorites=true, PATCH /api/v1/history/{id}	Same as History	Yes	Reuses History grid. Toggle favorite via PATCH.
13	History Look Detail	Full-screen viewer; compare original/result; favorite; delete	Route extra: HistoryItem + local providers	id, imageUrl, sourceImageUrl, isFavorite, createdAt	Yes	GET, PATCH, DELETE	GET /api/v1/history/{id}, PATCH /api/v1/history/{id}, DELETE /api/v1/history/{id}	Local/remote image URLs	Yes	Hero animation. Placeholder if extra null.
14	My Wardrobe	Grid of user's saved generated looks	historyListProvider (same local history)	Same as HistoryItem	Yes	GET	/api/v1/wardrobe/looks or reuse /api/v1/history	wardrobeSavedLookSamples defined but unused — shows real history	Yes	Currently identical to History data. Differentiate if "saved outfits" ≠ "all generations".
15	Settings	Account, notifications, language, theme, support, logout/delete	Local prefs + Firebase Auth	themeMode, notificationsEnabled, languageId, user session	Optional	GET, PATCH	/api/v1/users/me/preferences	Settings icons in assets/icons/ (local)	Yes (logout/delete)	Privacy/share URLs hardcoded (outfitchanger.app). FCM token synced to Firestore when signed in.
16	Account Info	Edit display name, photo, password	Firebase Auth + Firebase Storage	displayName, email (read-only), photoPath, photoUrl, password	No*	—	Firebase Auth + Storage	ic_edit_profile, ic_user, etc. (local)	Yes	Email not editable in UI.
17	Languages	Pick app locale (11 languages)	Hardcoded AppLanguagePresets +	languageId,	Optional	GET	/api/v1/languages	Flag PNGs in assets/images/languages/flags/ (+ light	No	Search/filter in UI

			SharedPreferences	hasSelectedLanguage, nativeName, englishName, flagUri				variants, local)		(client-side). First launch → login after save.
18	App Theme	System/light/dark picker	SharedPreferences (settings_theme_mode)	themeMode: system light dark	Optional	PATCH	/api/v1/users/me/preferences	Theme icons (local)	No	Can sync to cloud when user logged in.
19	About Us	Static app info	Hardcoded copy; version hardcoded 1.0.0 (Build 104)	appName, version, buildNumber, features[], supportEmail	Optional	GET	/api/v1/app/metadata	None (Material icons only)	No	Version mismatch: UI says Build 104, pubspec.yaml is 1.0.0+4. Should come from API or package info.

Asset Types Summary & Production Hosting Recommendations

Asset Type	Current Location	Used For	Count	Production Recommendation
Style catalog PNGs	assets/images/home/{men,women}/**	Virtual try-on, beauty lab, occasions, couple, presets, wardrobe regional/garment grids	500+ bundled PNGs	CDN (CloudFront/Cloudflare) or Firebase Storage + CDN — serve as thumbnailUrl / imageUrl in catalog API. Do not ship in APK long-term.
Home UI preview tiles	assets/images/home/ui/	Studio banner, Beauty Lab cards, occasion/couple preview rails	~20 PNGs	CDN — small marketing tiles; can stay bundled OR CMS-driven via home feed API.
Wardrobe banner/gallery	assets/images/wardrobe/	Wardrobe screen header/gallery	~10 PNGs	CDN or bundled (low churn).
Splash / Onboarding	assets/images/splash/, onboarding/	Launch & first-run	4 PNGs	Bundled OK (rarely changes) OR CMS via onboarding API.
Language flags	assets/images/languages/flags/ (+ light/)	Language picker	11 × 2 variants	Bundled OK OR flag CDN URLs in languages API.
Icons (UI chrome)	assets/icons/	Nav, auth, settings	~25 PNGs	Bundled — app chrome, not content.
Fonts	assets/fonts/outfit/Outfit-Variable.ttf	All typography	1 variable font	Bundled — standard for Flutter.
Lottie JSON	assets/animation/loading.json	Splash loading (defined, lightly used)	1 file	Bundled OR CDN if animated assets grow.
User photos	Device file system	Upload source, profile, history originals	Runtime	Firebase Storage or S3 — user-scoped paths users/{uid}/uploads/{id}.jpg.
Generated results	BFL temporary URLs → persisted locally	History, My Wardrobe, Try-On result	Runtime	S3 / Firebase Storage + signed URLs — persist after generation; never rely on BFL temp URLs long-term.
Audio	None	—	0	N/A
Remote JSON configs	None	—	0	Future: home feed, catalog metadata via REST.

- Recommended production stack:
- User uploads & generated looks: Firebase Storage or AWS S3 + CloudFront CDN
 - Catalog/style thumbnails: CDN (public or signed URLs) — biggest APK size win
 - App metadata / CMS content: Own REST API + CDN for media URLs
 - Auth: Keep Firebase Auth; backend verifies Firebase ID tokens

Models & Suggested JSON Structures

Freezed Models (2 — currently unused vs live BFL responses)

GenerateResponseModel
File: lib/features/try_on/data/models/generate_response_model.dart
App model today:

```
{ "job_id": "string", "status": "string" }
```

Actual BFL response (live):

```
{  "id": "uuid",  "polling_url": "https://api.bfl.ai/v1/get_result?id=...",  "status": "Pending"}
```

Suggested own-backend response:

```
{  "job_id": "uuid",  "polling_url": "/api/v1/try-on/jobs/uuid",  "status": "pending"}
```

ResultResponseModel
File: lib/features/try_on/data/models/result_response_model.dart
App model today:

```
{  "status": "processing | done | failed",  "result_image_url": "https://...",  "error": "string | null"}
```

Actual BFL poll response (live):

```
{  "status": "Ready | Pending | Processing | Error | Failed | Request Moderated",  "result": { "sample": "https://..." },  "details": { "message": "..." }}
```

Suggested own-backend poll response:

```
{  "status": "processing | done | failed",  "result_image_url": "https://cdn.example.com/results/uuid.jpg",  "error": null}
```

Domain Entities → Suggested API JSON

HistoryItem

```
{  "id": "uuid",  "created_at": "2026-09-17T12:00:00Z",  "image_url": "https://cdn.../result.jpg",  "source_image_url": "https://cdn.../source.jpg",  "is_favorite": false,  "style_id": "women_tops_03",  "category_id": "virtual_try_on"}
```

StyleCatalog (+ nested StyleItem, StyleTab)

```
{
  "category_id": "virtual_try_on",
  "title_key": "homeVirtualTryOn",
  "gender_scope": "both",
  "gender_tabs": [{ "id": "men", "title_key": "styleTabMen" }],
  "tabs": [{ "id": "tops", "title_key": "tryOnTabTops" }],
  "items": [{
    "id": "women_tops_01",
    "tab_id": "tops",
    "gender_tab_id": "women",
    "image_url": "https://cdn.../women_tops_01.png",
    "prompt_command": "wear this top exactly as shown"
  }]
}
```

HomeFeed (expand from current stub)

```
{
  "sections": [{
    "id": "beauty_lab",
    "title_key": "homeBeautyLab",
    "type": "category_cards",
    "items": [{
      "category_id": "hair_styles",
      "thumbnail_url": "https://cdn.../beauty_hair_styles.png",
      "label_key": "homeHairStyles"
    }]
  }, {
    "id": "occasions",
    "type": "image_rail",
    "category_id": "occasions",
    "items": [{ "style_id": "occasions_casual_women_01", "thumbnail_url": "..."} ]
  }]
}
```

WardrobeCategory / Section

```
{
  "id": "chinese_traditional",
  "title_key": "wardrobeChineseTraditional",
  "browse_tab_id": "chinese",
  "background_token": "chinese_traditional_bg",
  "preview_items": [{ "style_id": "chinese_01", "thumbnail_url": "..."} ]
}
```

AuthUser (from Firebase; optional sync endpoint)

```
{
  "id": "firebase_uid",
  "email": "user@example.com",
  "display_name": "Atif",
  "photo_url": "https://storage.../avatar.jpg",
  "is_guest": false
}
```

SettingsState / User Preferences

```
{
  "theme_mode": "system | light | dark",
  "notifications_enabled": true,
  "language_id": "en_US",
  "style_gender_preference": "men | women"
}
```

AppLanguage

```
{
  "id": "en_US",
  "language_code": "en",
  "country_code": "US",
  "native_name": "English",
  "english_name": "English",
  "flag_url": "https://cdn.../flags/us.png"
}
```

Try-On Generate Request (backend proxy)

```
{
  "source_image_base64": "data:image/jpeg;base64,...",
  "style_id": "women_tops_03",
  "category_id": "virtual_try_on",
  "style_reference_image_base64": "data:image/jpeg;base64,...",
  "prompt": "optional user text",
  "width": 768,
  "height": 1024
}
```

Features Using Local/Mock Data — Must Migrate to Real APIs

Feature	Current Source	Migration Priority	Target API
Style catalogs (all categories)	StyleCatalogStore hardcoded ~500 PNGs	P0 — Critical	GET /api/v1/catalog/{categoryId}
Home feed (Beauty Lab, Occasions, Couple rails)	Hardcoded widgets + empty HomeRemoteDataSource	P0	GET /api/v1/home/feed
Wardrobe categories	WardrobeCatalog hardcoded	P0	GET /api/v1/wardrobe/categories
History list	SharedPreferences only	P0	GET/PATCH/DELETE /api/v1/history
My Wardrobe	Same as history (local)	P1	GET /api/v1/wardrobe/looks or alias history
Favorites	Local filter on history	P1	GET /api/v1/history?favorites=true
Try-On generation	Direct BFL call (works but no user/job tracking)	P1	Proxy via POST /api/v1/try-on/generate
Generated image persistence	BFL temp URL → local file	P1	Upload to S3/Storage on job complete
Onboarding pages	Hardcoded 3 pages	P2	GET /api/v1/onboarding/pages
Languages list	Hardcoded 11 languages	P2	GET /api/v1/languages (optional)
About / app metadata	Hardcoded version text	P2	GET /api/v1/app/metadata
User preferences sync	SharedPreferences only	P2	GET/PATCH /api/v1/users/me/preferences
Auth	Firebase (keep)	No migration	Verify Firebase ID token on backend

Recommended RESTful API Structure

Base URL & Versioning

```
Production: https://api.outfitchanger.app/api/v1
Staging:    https://staging-api.outfitchanger.app/api/v1
```

Auth Headers

```
Authorization: Bearer {firebase_id_token} // user-scoped routes
X-API-Key: {server_api_key} // optional app attestation
Content-Type: application/json
```

Route Map

Method	Endpoint	Description
App		
GET	/app/metadata	Version, support links, feature flags
GET	/onboarding/pages	CMS onboarding content
GET	/languages	Supported locales + flag URLs
Home & Catalog		
GET	/home/feed	Home sections + rail items (?gender=)
GET	/catalog/{categoryId}	Full style catalog (?tab=&gender=)
GET	/wardrobe/categories	Wardrobe regional sections + previews
Try-On / AI		
POST	/try-on/generate	Start generation job (proxies BFL)
GET	/try-on/jobs/{jobId}	Poll job status + result URL
DELETE	/try-on/jobs/{jobId}	Cancel in-flight job
History & Wardrobe		
GET	/history	List looks (?favorites=&page=&limit=)
POST	/history	Save new look after generation
GET	/history/{id}	Single look detail
PATCH	/history/{id}	Update is_favorite
DELETE	/history/{id}	Delete one look
DELETE	/history	Bulk delete ({ "ids": ["..."] })
GET	/wardrobe/looks	Saved looks (alias or subset of history)
User		
GET	/users/me	Profile + preferences
PATCH	/users/me	Update display name
PATCH	/users/me/preferences	Theme, language, gender, notifications
POST	/users/me/avatar	Upload profile photo (multipart)
DELETE	/users/me	Delete account (or delegate to Firebase)
POST	/users/me/fcm-token	Register push token (today: Firestore direct)

Naming Conventions

- `snake_case` in JSON bodies (created_at, image_url, is_favorite)
- kebab-case or snake_case in URLs (pick one; recommend kebab: `/try-on/jobs`)
- Plural nouns for collections (`/history` , `/catalog/{id}`)
- Pagination: `?page=1&limit=20` → response { "data": [], "meta": { "page", "limit", "total" } }
- Errors: { "error": { "code": "network", "message": "Human-readable message" } }

Standard Response Envelope

```
{
  "data": { },
  "meta": { "page": 1, "limit": 20, "total": 142 },
  "error": null
}
```

Existing External APIs (Not Your Backend — Reference Only)

Service	Base URL	Endpoints	Auth
BFL FLUX	https://api.bfl.ai	POST /v1/flux-2-pro, GET {polling_url}	Header x-key: API_KEY
Firebase Auth	Firebase SDK	signIn, signUp, Google, anonymous, reset	Firebase config
Firebase Storage	Firebase SDK	users/{uid}/avatar.jpg	Firebase rules
Firestore	Firebase SDK	users/{uid} → fcmToken	Firebase rules

Key Gaps for Backend Developer

- ApiClient is wired but unused — implement the stub endpoints in HomeRemoteDataSource, HistoryRemoteDataSource, WardrobeRemoteDataSource.
- Catalog is the largest migration — ~500 bundled PNGs should become CDN URLs returned by `/catalog/{categoryId}`.
- History has no cloud sync — sign-out clears local data via `UserSessionCleanup`; users lose history on new device.
- BFL models don't match live API — align `GenerateResponseModel` / `ResultResponseModel` or delete unused code.
- Try-On should be proxied — hide BFL API key server-side; track jobs per user; persist results to your storage.
- My Wardrobe vs History — currently same data; clarify product intent before separate endpoints.